CampusConnect: Campus Placement Management Portal

Project Proposal for UCS503P
Submitted to: Nisha Thakur
Thapar Institute of Engineering and Technology

Mayank Garg, COPC*
Paryag Bansal, COPC†
Aashana, COPC‡

August 14, 2026

1 Higher-order goal

. . . secondary framing

The goal of CampusConnect is to make the campus placement process easier for students and placement administrators. Students should be able to see suitable placement drives, complete their profile, and track their applications in one place. The main engineering goal is to build a working web application that makes placement information and application status easier to manage.

2 Time-to-value

. . . primary heuristic

- We will first build the main student flow: register, log in, view placement drives, and check application status.
- We will take feedback from a small group of students after each weekly update and make simple improvements quickly.
- The first version will focus on useful screens and a working backend instead of adding too many features at once.

3 Problem Statement

Students often get placement information from different sources such as emails, WhatsApp groups, notices, and messages from classmates. This can make it difficult to know which drives are open and whether they are eligible. Common problems are:

- **Scattered information:** drive details, deadlines, and updates are shared in many places.
- **Unclear eligibility:** students may not know if their CGPA, course, graduation year, or backlogs meet a company's rules.

*Roll No: 1024170442, Email: mgarg7_be24@thapar.edu
†Roll No: 1024170446, Email: pbansal5_be24@thapar.edu
‡Roll No: 1024170427, Email: abhandari_be24@thapar.edu

- **No simple tracking:** students may not know the current status of their application after applying.
- **Manual admin work:** placement administrators need to manage student details, applications, and documents separately.

Why this matters now (contextual relevance): A single portal can save time for students and reduce repeated work for the placement cell, especially when several companies open drives at the same time.

4 Proposed Solution *... Software Proposition*

4.1 Overview

Build a **web-based** campus placement portal called CampusConnect with the following parts:

- **Student account:** students can register using their Thapar email, log in, and maintain their profile.
- **Placement drive view:** students can see available companies, job roles, packages, deadlines, locations, and eligibility.
- **Application tracking:** students can apply for drives and view the status of their applications.
- **Document and profile support:** students can add academic details, skills, projects, certificates, and resume information.
- **Admin dashboard:** administrators can manage students, check documents, and review applications.

4.2 Core workflow

1. A student registers with a Thapar email address and logs in to CampusConnect.
2. The student completes profile details such as course, graduation year, CGPA, skills, and resume.
3. The student views placement drives and checks the basic eligibility information.
4. The student applies for a suitable drive and follows the application status from the dashboard.
5. The admin checks student details and documents, then manages applications and drive-related information.

4.3 Operational constraints for an educational, tier-2/3 setting

- The portal should work well on laptops and phones through a responsive web interface.
- The first version will use simple rules for eligibility and profile checks instead of complex algorithms.
- The project will use commonly available web tools so it can be deployed and tested without costly infrastructure.

5 Solution Approach *... Engineering focus*

This is an engineering course project, so the focus is on making the placement workflow clear, usable, and reliable. The project does not try to create a new placement algorithm. It focuses on building the portal and connecting its student and admin functions properly.

5.1 Duplicate/quality assistance *... non-research*

Profile and application quality checks will use simple rules:

- Check that important fields such as email, course, graduation year, and CGPA are present before an application is submitted.
- Use the registered email as a unique student identifier to avoid duplicate accounts.
- Show missing profile fields to the student before they apply, instead of automatically rejecting their application.

5.2 Web architecture *... web-based solution*

A simple three-layer web architecture will be used:

- **Frontend:** React and Vite will provide the student login, student dashboard, and admin pages.
- **Backend API:** Node.js and Express will handle registration, login, user data, and future placement APIs.
- **Database:** MongoDB with Mongoose will store user accounts, academic details, skills, projects, resumes, and role information.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Build the React frontend and check the code on every important change.
- Run basic backend tests for registration, login, and validation rules.
- Make it easier to deploy a tested version to a staging environment.

6 Evaluation Criterion *... measurable and attributable*

The project will be checked using simple measures linked to the main student workflow.

6.1 Primary evaluation metric

Application completion time (ACT): the time a student needs to log in, complete the required profile fields, find a suitable drive, and submit an application.

- Target: most pilot users should be able to complete this flow in 5 minutes or less when their details are ready.
- Attribution: measure the time from login to the application confirmation screen during a small user test.

6.2 Secondary evaluation metrics

- **Profile completion rate:** percentage of pilot students who complete the required profile fields.
- **Application clarity:** percentage of pilot students who can correctly identify their application status.
- **Admin task time:** time needed to find a student record or review an application in the admin dashboard.
- **Reliability:** successful registration and login attempts during the pilot period.

6.3 Pilot validation plan *. . . high-level*

- Ask 10–15 students to register, complete a profile, and use a sample placement drive.
- Ask 2–3 users to try the admin pages with sample student and application data.
- Collect short feedback about ease of use, missing information, and confusing steps.

7 Scalability *. . . with foresight*

The portal should be able to grow from a small student test to wider use in the institute.

1. **Scalable (theoretically):** The system can support more students, drives, and applications by:
   - keeping the frontend and backend separate,
   - adding pagination and database indexes for student and application lists,
   - keeping API routes independent of the user interface.
2. **Operationally deployable (practically):** The system can run on normal web hosting using:
   - a managed MongoDB database,
   - a platform-hosted frontend and backend,
   - a simple CI/CD workflow for staging and production updates.

8 Engine availability heuristic

The project can be built with mature tools that are already widely used for web applications:

- React and Vite for the frontend interface.
- Node.js and Express for REST APIs.
- MongoDB and Mongoose for storing data.
- bcrypt for password hashing and JSON Web Tokens for authentication.
- GitHub Actions or a similar CI tool for build and test checks.
- A cloud platform such as Render or Vercel for deployment.

Using these tools lets us focus on the placement workflow and user experience instead of building basic infrastructure from scratch.

9 Project scope and deliverables *. . . iteration-friendly*

9.1 Initial deliverable *. . . first iteration*

- Student registration and login with Thapar email validation.
- Student profile with academic details, skills, projects, certificates, and resume fields.
- Student dashboard showing profile completion, eligible drives, and applications.
- Admin pages for student management, document verification, and applications.
- Basic backend APIs for user registration and login.
- Build, lint, and basic test setup for the project.

9.2 Subsequent deliverables

- Create and manage placement drives from the admin dashboard.
- Add application submission and application-status updates.
- Add rule-based eligibility checks using student profile data.
- Add search, filters, and notifications for drives and applications.

10 Risks and mitigations

- **Student data privacy:** store only needed profile data, hash passwords, and limit admin access.
- **Incomplete profiles:** clearly show which fields are missing before students apply.
- **Wrong eligibility data:** keep eligibility rules visible and allow admins to review applications.
- **Deployment issues:** use environment variables for secrets and test the project in staging before release.

11 Summary

CampusConnect is a web portal for handling the basic campus placement process in one place. It will help students maintain their profile, view placement drives, and track applications. It will also help administrators manage student details, documents, and applications. The project uses a practical React, Node.js, Express, and MongoDB setup and will be developed in small working steps with feedback from students.